

COURSEWORK & STUDY GUIDE

System Design

From First Principles to Planet Scale

Every concept explained in depth — networking, databases, distributed systems, microservices, reliability, and 20 case studies

7 Phases · 50+ Concepts · Complete Reference

PHASE 1

Networking & Protocols

1.1 The OSI & TCP/IP Models

Every network interaction passes through a stack of abstraction layers. Understanding where each protocol lives prevents you from designing systems that fight the network rather than work with it.

OSI MODEL — 7 LAYERS (TOP TO BOTTOM)

Layer 7 – Application	HTTP, DNS, SMTP, WebSocket – what your code sees
Layer 6 – Presentation	TLS/SSL, encoding, compression
Layer 5 – Session	Session management, keep-alive
Layer 4 – Transport	TCP (reliable), UDP (fast) – ports live here
Layer 3 – Network	IP routing, ICMP – addresses live here
Layer 2 – Data Link	Ethernet, MAC addresses – local delivery
Layer 1 – Physical	Cables, fibre, radio waves

Why This Matters for Design

Load balancers at Layer 4 route by IP+port only — fast but dumb. Layer 7 load balancers can inspect HTTP headers, cookies, and paths — slower but much smarter. CDNs operate at Layer 7. When you choose a load balancer, you are choosing a layer.

1.2 TCP vs UDP

Choosing between TCP and UDP is one of the earliest decisions in any networked system design.

Property	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless — fire and forget
Reliability	Guaranteed delivery, retransmits lost packets	No guarantee — packets can be lost
Ordering	In-order delivery guaranteed	Packets may arrive out of order
Overhead	Higher (headers, ACKs, congestion control)	Minimal (8-byte header)
Latency	Higher due to acknowledgements	Lower

Use cases	HTTP, databases, file transfer, email	Video streaming, gaming, DNS, VoIP
-----------	---------------------------------------	------------------------------------

Design Insight

HTTP/3 (QUIC) runs over UDP and implements its own reliability layer. This gives it TCP's reliability with better performance — especially on mobile where TCP's connection migration is costly. Most modern CDNs support HTTP/3.

1.3 DNS — How Names Become IPs

DNS is not just name resolution — it is also a primitive load balancer, a failover mechanism, and the entry point into every CDN. Understanding DNS resolution is essential for designing global systems.

DNS RESOLUTION FLOW

```
Browser checks local cache
  |
  v (cache miss)
OS resolver → Recursive Resolver (your ISP or 8.8.8.8)
  |
  v (cache miss)
Root Name Server → tells resolver which TLD server (.com, .io, etc.)
  |
  v
TLD Name Server → tells resolver which Authoritative Name Server
  |
  v
Authoritative Name Server → returns the actual IP address
  |
  v (cached at each layer for TTL duration)
Browser gets the IP — makes TCP connection
```

Record Type	Purpose	Example
A	IPv4 address	api.example.com → 1.2.3.4
AAAA	IPv6 address	api.example.com → 2001:db8::1
CNAME	Alias to another domain	www → example.com
MX	Mail server for domain	example.com → mail.example.com
TXT	Arbitrary text (SPF, DKIM, verification)	v=spf1 include:...
NS	Which name servers are authoritative	example.com NS → ns1.dns.com

SRV	Service location (host, port, priority)	Used by gRPC, SIP, etc.
-----	---	-------------------------

DNS as Load Balancer

DNS round-robin returns multiple A records and rotates through them. Weighted routing sends more traffic to larger instances. Latency-based routing returns the nearest server. Health-check based failover removes unhealthy endpoints. All of this is DNS, not code.

1.4 HTTP — Evolution of the Web Protocol

Version	Key Feature	Problem Solved
HTTP/1.0	One request per TCP connection	Simple but extremely inefficient
HTTP/1.1	Keep-alive, pipelining	Reuses connections — but head-of-line blocking remains
HTTP/2	Multiplexing, header compression, server push	Multiple streams over one TCP connection — faster
HTTP/3 (QUIC)	UDP-based, 0-RTT, connection migration	Eliminates TCP head-of-line blocking, better on mobile

HTTP/2 Multiplexing — The Key Improvement

HTTP/1.1 VS HTTP/2

HTTP/1.1 (3 requests need 3 connections or must queue):

```
Conn 1: [Request A —————] [Response A ———]
Conn 2: [Request B —————] [Response B ———]
Conn 3: [Request C —————] [Response C ———]
```

HTTP/2 (3 requests over 1 connection, multiplexed):

```
Conn 1: [Req A][Req B][Req C][Resp B][Resp A][Resp C]
              (out-of-order is fine)
```

REST vs GraphQL vs gRPC

	REST	GraphQL	gRPC
Transport	HTTP/1.1 or 2	HTTP/1.1 or 2	HTTP/2
Format	JSON (usually)	JSON	Protocol Buffers (binary)
Schema	OpenAPI/Swagger	SDL (typed)	.proto files (strongly typed)
Over/Under-fetching	Common problem	Solved by design	N/A — explicit methods
Streaming	Limited (SSE)	Subscriptions	Native bidirectional

Browser support	Native	Native	Requires grpc-web proxy
Best for	Public APIs, CRUD	Complex frontends, mobile	Internal services, low latency

1.5 Load Balancing

A load balancer distributes incoming requests across multiple backend servers. It is the entry point into any scaled system and must itself be highly available.

Algorithm	How It Works	Best For
Round Robin	Requests cycle through servers 1,2,3,1,2,3	Homogeneous servers, stateless services
Least Connections	New request goes to server with fewest active connections	Long-lived connections, variable request duration
IP Hash	Client IP hashed to always reach the same server	Sticky sessions without session store
Weighted Round Robin	Servers get requests proportional to their weight	Heterogeneous hardware (bigger servers get more)
Random	Randomly select a server	Simple, surprisingly effective with enough servers
Least Response Time	Route to server with lowest average response time	Latency-sensitive applications

LAYER 4 VS LAYER 7 LOAD BALANCING

Layer 4 (Transport) Load Balancer:

- Client → LB (sees IP:port only) → Backend
- Faster (no packet inspection)
- Cannot route by URL path, headers, or cookies
- Example: AWS NLB, HAProxy in TCP mode

Layer 7 (Application) Load Balancer:

- Client → LB (inspects HTTP) → Backend
- Can route /api/* to API servers, /static/* to CDN
- Can read cookies for sticky sessions
- Can terminate TLS (offload crypto from backends)
- Slightly slower, much more flexible
- Example: AWS ALB, Nginx, HAProxy in HTTP mode

PHASE 2

Scalability

2.1 Vertical vs Horizontal Scaling

	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
What	Bigger machine (more CPU, RAM, SSD)	More machines (add servers)
Limit	Hard ceiling — biggest machine available	Theoretically unlimited
Complexity	Low — one machine, no distribution	High — distributed system challenges
Cost	Exponential — hardware prices jump at high end	Linear — commodity hardware
Downtime	Required to upgrade hardware	Can add nodes without downtime
Failure	Single point of failure	Redundant — some nodes can fail
Best for	Databases (easier), early-stage products	Web servers, cache layers, stateless services

Practical Reality

Most systems do both. Databases are often vertically scaled first (bigger RDS instance) because horizontal scaling requires sharding which is complex. Stateless web servers and cache nodes are horizontally scaled because they're easy to replicate. Start vertical, move horizontal as you grow.

2.2 Caching — The Most Important Scalability Tool

Every order-of-magnitude improvement in read throughput comes from caching. A cache is a fast, often in-memory store that keeps frequently accessed data close to where it is needed.

Cache Locations (from closest to farthest from the user)

CACHE LAYERS IN A TYPICAL SYSTEM

1. Browser Cache — HTTP Cache-Control headers, no network round trip
2. CDN / Edge Cache — Nearest POP, reduces request to origin

- 3. API Gateway Cache – Cache by URL/headers, reduces backend calls
- 4. Application Cache – In-process (LRU hashmap), sub-millisecond
- 5. Distributed Cache – Redis/Memcached, shared across instances
- 6. Database Query Cache – Query result caching (often disabled in modern DBs)
- 7. Database Buffer Pool – DB engine caches pages in RAM
- 8. OS Page Cache – OS caches filesystem reads in RAM

Caching Strategies

Strategy	How It Works	Best For	Downside
Cache-Aside (Lazy)	App checks cache first; on miss, loads from DB and populates cache	General-purpose reads	Initial cache miss is slow; stale data possible
Read-Through	Cache sits in front of DB; cache fetches from DB on miss automatically	Simpler app code	Cache is on critical path; cold start is slow
Write-Through	On write, update both cache and DB synchronously	Strong consistency, read-heavy	Slower writes; cache polluted with rarely-read data
Write-Behind (Write-Back)	On write, update cache only; flush to DB asynchronously	Write-heavy, high throughput	Risk of data loss if cache crashes before flush
Refresh-Ahead	Pre-emptively refresh cache before TTL expires	Predictable access patterns	Wastes resources if prediction is wrong

Cache Eviction Policies

Policy	Evicts	Best For
LRU — Least Recently Used	The item not accessed for the longest time	General-purpose: recency predicts future use
LFU — Least Frequently Used	The item accessed fewest times	Popularity-driven workloads
TTL — Time-To-Live	Items older than a configured duration	Data with known freshness requirements
FIFO — First In, First Out	The oldest item by insertion time	Simple queues, not great for general caching
Random	A random item	Surprisingly effective for large caches

The Thundering Herd Problem

When a popular cache key expires, hundreds of requests simultaneously miss the cache and all hit the database at the same time. Solutions: (1) mutex/lock — only one request fetches, others wait; (2) probabilistic early expiry — randomly refresh before TTL expires; (3) background refresh — serve stale while refreshing; (4) never expire — rely only on invalidation events.

2.3 Database Scaling

Read Replicas

PRIMARY-REPLICA REPLICATION

```

All Writes → [ Primary DB ] → WAL replication → [ Replica 1 ]
                                                ↳ [ Replica 2 ]
                                                ↳ [ Replica 3 ]
  
```

```

All Reads → [ Load Balancer ] → any replica
  
```

Replication lag: typically milliseconds, but can grow under load.
 Read your own writes: route writes AND immediate reads to primary.

Database Sharding

Sharding horizontally partitions data across multiple database instances. Each shard holds a subset of the data and is completely independent.

Strategy	How	Pro	Con
Hash Sharding	shard = hash(userId) % num_shards	Even distribution	Resharding requires moving half the data
Range Sharding	shard based on value ranges (A-M, N-Z)	Range queries are fast	Hot spots if data isn't uniform
Directory Sharding	Lookup table maps each key to its shard	Flexible, no hotspots	Lookup table is a bottleneck and SPOF
Geo Sharding	Users in EU → EU shard, US → US shard	Low latency, data residency	Uneven load if regions have different sizes

The Resharding Problem

The biggest challenge with hash sharding is adding a new shard. Changing num_shards from 4 to 5 changes where every key maps. Consistent hashing solves this by only remapping 1/N of keys when adding a node — used by Cassandra, Redis Cluster, and DynamoDB.

2.4 Rate Limiting Algorithms

Algorithm	How It Works	Pros	Cons
-----------	--------------	------	------

Fixed Window	Count requests per fixed time window (per minute)	Simple to implement	Burst allowed at window boundary (double the limit)
Sliding Window Log	Log timestamp of each request; count requests in last N seconds	Accurate, no boundary burst	High memory — stores every request timestamp
Sliding Window Counter	Approximate sliding window using two fixed windows weighted by overlap	Memory efficient, no boundary burst	Slightly approximate
Token Bucket	Tokens added at fixed rate; each request consumes one token	Allows controlled bursts; smooth throttle	Slightly complex to implement distributed
Leaky Bucket	Requests enter a queue (bucket); processed at fixed rate	Enforces smooth output rate	Queue can delay time-sensitive requests

TOKEN BUCKET ALGORITHM

Bucket capacity: 10 tokens
Refill rate: 2 tokens/second

```
t=0s: bucket=[10 tokens] → request arrives → consume 1 → [9 tokens] → ALLOW
t=0s: bucket=[9 tokens] → 5 rapid requests → consume 5 → [4 tokens] → ALLOW
t=0s: bucket=[4 tokens] → 5 rapid requests → consume 4 → [0 tokens] → ALLOW
      last request → 0 tokens remaining → REJECT (429 Too Many Requests)
t=1s: refill → [2 tokens] → 1 request → ALLOW
```

PHASE 3

Databases & Storage

3.1 SQL vs NoSQL — The Real Distinction

The SQL vs NoSQL debate is often framed as 'relational vs non-relational' but the more useful lens is: what consistency and query guarantees does your system need?

Dimension	SQL (RDBMS)	NoSQL
Data Model	Tables with rows and typed columns	Documents, key-value, wide-column, graph
Schema	Fixed, enforced schema	Flexible or schema-less
Transactions	Full ACID across multiple tables	Typically ACID within one document/partition
Query Language	SQL — powerful, joins, aggregations	Limited — no joins, restricted queries
Scaling Model	Primarily vertical; sharding is hard	Designed for horizontal scale
Consistency	Strong by default	Tunable: eventual to strong
Best for	Complex relationships, reporting, CRUD	High-scale, simple access patterns, flexible schema

3.2 ACID Transactions

ACID is the gold standard of database reliability. Understanding each property helps you know what you're giving up when you choose a database that doesn't provide full ACID.

Property	Meaning	Example
Atomicity	A transaction is all-or-nothing. If any step fails, the entire transaction is rolled back.	Transfer \$100: debit account A AND credit account B. If credit fails, debit is undone.
Consistency	A transaction moves the database from one valid state to another. All constraints remain satisfied.	A bank transfer can't leave a negative balance if a NOT NULL constraint requires positive balance.
Isolation	Concurrent transactions don't see each other's uncommitted	Two users booking the last seat in an event shouldn't both succeed.

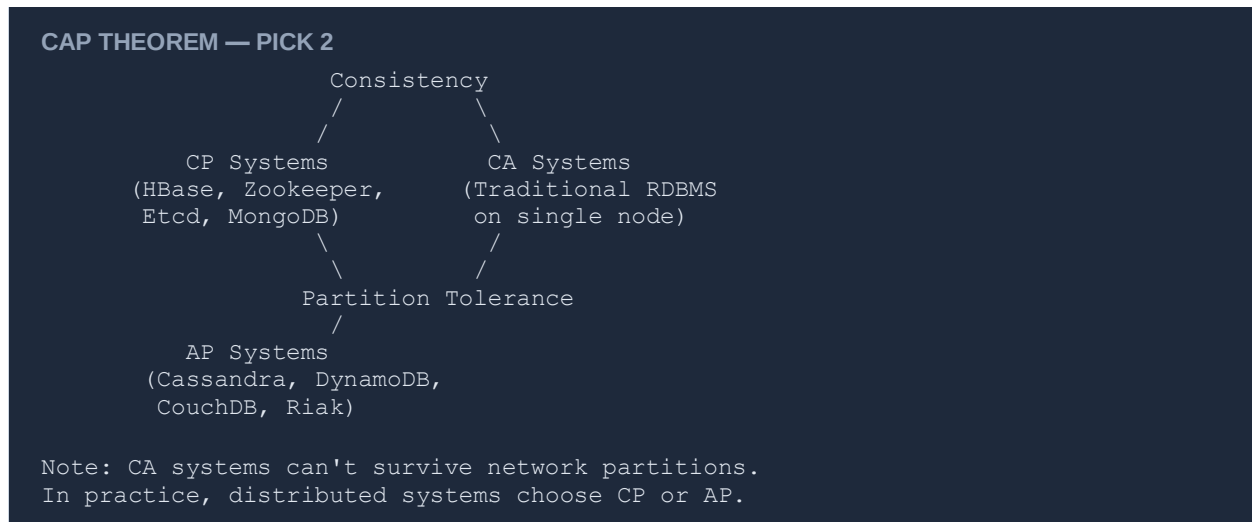
	changes. Levels: read uncommitted, read committed, repeatable read, serialisable.	
Durability	Once committed, a transaction persists even if the server crashes immediately after.	Write-ahead log (WAL) ensures committed data survives a crash.

Isolation Levels and Their Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
Read Uncommitted	Possible	Possible	Possible	Fastest
Read Committed	Prevented	Possible	Possible	Fast (default in PostgreSQL)
Repeatable Read	Prevented	Prevented	Possible	Moderate (default in MySQL)
Serialisable	Prevented	Prevented	Prevented	Slowest

3.3 CAP Theorem

In a distributed system, you can only guarantee two of three properties simultaneously: Consistency, Availability, and Partition Tolerance. Since network partitions are inevitable, you always choose between C and A.



System Type	Trade-off	Example Systems	Use When
-------------	-----------	-----------------	----------

CP	Returns error or timeout if it cannot guarantee consistency	HBase, ZooKeeper, etcd, MongoDB (default config)	Financial data, leader election, config management
AP	Returns potentially stale data rather than an error	Cassandra, DynamoDB, CouchDB	Shopping carts, user profiles, DNS, social feeds

Beyond CAP — PACELC

PACELC extends CAP: even when the system is running normally (no partition), you trade off Latency vs Consistency. A system that chooses strong consistency has higher write latency because it must wait for all replicas to acknowledge. This is why DynamoDB lets you choose between eventual and strong consistency per read.

3.4 Database Indexing In Depth

An index is a separate data structure that the database maintains to speed up queries. Indexes make reads faster but writes slower (because every write must also update indexes). Choosing the right indexes is one of the most impactful performance decisions you can make.

Index Type	Structure	Best For	Limitation
B-Tree (default)	Balanced tree, sorted order	Range queries, equality, ORDER BY, most queries	Not good for low-cardinality columns (e.g. boolean)
Hash	Hash table	Exact equality only (=)	Cannot do range queries (>, <, BETWEEN)
Composite	B-tree on multiple columns	Queries filtering on multiple columns together	Column order matters — leftmost prefix rule
Covering	Index includes all columns in the query	Queries answered entirely from index (no table scan)	Larger index size
Partial	Index only rows matching a condition	Sparse data (e.g. index only active=true rows)	Only useful for that specific condition
Full-Text	Inverted index of word positions	Text search (LIKE '%word%' won't use B-tree)	Not suitable for exact or range queries

HOW A B-TREE INDEX WORKS

```
Table: users (id, name, email, created_at)
Query: SELECT * FROM users WHERE email = 'alice@example.com'
```

Without index: Full table scan — $O(n)$, reads every row

```
With index on (email):
  Root node: [g] [p]
    /       |       \
  [a-g]    [h-p]    [q-z]
  / \      ...      ...
[alice] → row pointer → fetch row from heap

With index: O(log n) — reads only nodes along the path

EXPLAIN ANALYSE output — what to look for:
Seq Scan → no index used (bad for large tables)
Index Scan → index used to find rows, then heap fetch
Index Only Scan → covering index, no heap fetch (fastest)
```

3.5 Consistent Hashing

Consistent hashing is the algorithm that makes horizontal scaling of distributed caches and databases practical. When you add or remove a node, only a fraction of keys need to be remapped — not all of them.

CONSISTENT HASHING RING

Imagine a ring of hash values from 0 to $2^{32}-1$.

Nodes placed at positions by hashing their names:

```
hash('Server-A') = 12% → placed at position 12%
hash('Server-B') = 45% → placed at position 45%
hash('Server-C') = 78% → placed at position 78%
```

Keys placed by hashing and walking clockwise to next node:

```
hash('user:123') = 20% → goes to Server-B (next clockwise)
hash('user:456') = 60% → goes to Server-C
hash('user:789') = 90% → wraps around to Server-A
```

Add Server-D at position 55%:

Only keys between 45% and 55% (previously on Server-C) move.
All other keys are unaffected.

Virtual nodes: each physical server gets 150+ virtual positions on the ring. This ensures even distribution even with few nodes.

PHASE 4

Distributed Systems

4.1 Replication Strategies

Strategy	Description	Consistency	Availability	Use Case
Single Leader	One primary accepts writes; replicas receive updates	Strong (sync) or eventual (async)	Failover needed if primary dies	PostgreSQL, MySQL, MongoDB default
Multi-Leader	Multiple nodes accept writes; sync to each other	Eventual — write conflicts possible	High write availability	Multi-region active-active, CRDTs
Leaderless (Dynamo-style)	Any node accepts writes; quorum determines validity	Tunable via $W+R > N$ quorum	Very high — no single point of failure	Cassandra, DynamoDB, Riak

Quorum Reads and Writes

QUORUM CONSENSUS (N=3 REPLICAS)

N = total replicas = 3

W = write quorum (must acknowledge write) = 2

R = read quorum (must respond to read) = 2

Condition for strong consistency: $W + R > N \rightarrow 2 + 2 > 3 \checkmark$

Strong consistency (W=2, R=2):

Write: 2/3 nodes must acknowledge $\rightarrow$ slower writes, guaranteed read-your-writes

Read: 2/3 nodes respond $\rightarrow$ at least 1 will have the latest write

High availability (W=1, R=1):

Fastest operations, but reads may return stale data

W=3 (write to all):

Slowest writes (must wait for all nodes), fastest reads (R=1 is always fresh)

4.2 Message Queues & Event Streaming

Message queues decouple services — the producer does not wait for the consumer to process the message. This enables independent scaling, absorbs traffic spikes, and allows services to fail independently.

Property	Traditional Queue (RabbitMQ/SQS)	Event Stream (Kafka)
Message lifetime	Deleted after consumption	Retained for configurable period (days/forever)
Replay	No — consumed messages are gone	Yes — consumers can rewind to any offset
Consumer model	Competing consumers — one consumer per message	Consumer groups — each group gets all messages
Ordering	Per-queue FIFO (with caveats)	Per-partition ordering guaranteed
Throughput	Moderate (thousands/sec)	Very high (millions/sec)
Use case	Task queues, work distribution, one-time jobs	Event sourcing, audit logs, data pipelines, CDC

Delivery Semantics

AT-MOST-ONCE VS AT-LEAST-ONCE VS EXACTLY-ONCE

At-Most-Once:

Producer sends → Consumer acknowledges BEFORE processing
If consumer crashes after ack but before processing: message lost
Use: logging, analytics where occasional loss is acceptable

At-Least-Once:

Producer sends → Consumer processes → THEN acknowledges
If consumer crashes after processing but before ack: message replayed
Consumer MUST be idempotent (safe to process twice)
Use: most business operations (payments, emails with deduplication)

Exactly-Once:

Achieved via idempotency keys + deduplication at consumer
OR via transactional producers (Kafka transactions)
Use: financial transactions, critical state changes
Note: true exactly-once is expensive — verify you really need it

4.3 The Saga Pattern — Distributed Transactions

In a microservices architecture, a single business operation (e.g. 'place an order') spans multiple services. You cannot use a database transaction across service boundaries. The Saga pattern manages this.

SAGA PATTERN — ORDER PLACEMENT

Choreography-based Saga (event-driven, decentralised):

```
Order Service      → emits OrderCreated event
Payment Service    → listens, charges card → emits PaymentCompleted
Inventory Service  → listens, reserves stock → emits StockReserved
Shipping Service   → listens, creates shipment → emits ShipmentCreated
```

```
Failure (payment fails):
Payment Service    → emits PaymentFailed
Order Service      → listens, cancels order → emits OrderCancelled
```

Orchestration-based Saga (central coordinator):

```
Saga Orchestrator → call Payment Service
Payment Service   → returns success
Saga Orchestrator → call Inventory Service
Inventory Service → returns failure (out of stock)
Saga Orchestrator → call Payment Compensate (refund)
Saga Orchestrator → mark order as FAILED
```

The Outbox Pattern

The most reliable way to publish events is the Outbox Pattern: instead of publishing an event directly to Kafka, write it to an 'outbox' table in the same database transaction as your business write. A separate process reads the outbox table and publishes to Kafka. This ensures the event is only published if the database write succeeds — no dual-write inconsistency.

4.4 Idempotency

An operation is idempotent if performing it multiple times has the same effect as performing it once. In distributed systems, at-least-once delivery means your consumers will sometimes process the same message twice. Making operations idempotent is not optional.

IDEMPOTENCY KEY PATTERN

Client generates a unique idempotency key for each request:

```
POST /payments
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
{ amount: 100, currency: 'USD' }
```

Server logic:

1. Check if idempotency key exists in payments table
- 2a. If NO: process payment, store result with idempotency_key
- 2b. If YES: return the previously stored result (do NOT charge again)

This makes retrying safe — client can retry on network error without worrying about double-charging.

Key expiry: idempotency keys should expire after 24-48 hours.
The client should use the same key for retries within a request window.

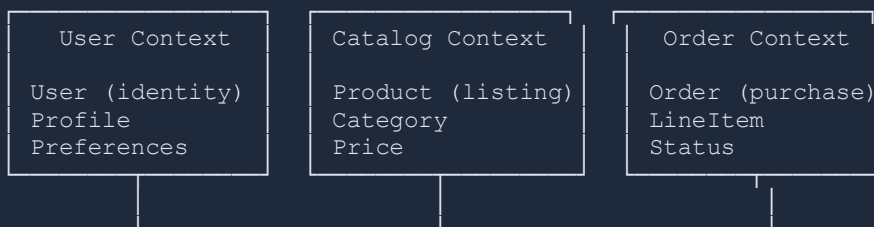
PHASE 5

Microservices & API Design

5.1 Service Decomposition with DDD

Domain-Driven Design (DDD) provides a framework for deciding where to draw service boundaries. The core concept is the Bounded Context — a cohesive subdomain where a single model applies.

E-COMMERCE BOUNDED CONTEXTS



Anti-Corruption Layers
(each context translates foreign models)

Note: 'Product' means different things in each context.
In Catalog: name, description, category, images.
In Order: price at time of purchase, quantity, SKU.
These should be separate models, not shared.

5.2 Resilience Patterns

Circuit Breaker

CIRCUIT BREAKER STATE MACHINE

```
CLOSED (normal operation)
├─ Request passes through to downstream service
├─ Failure rate tracked over rolling window
├─ If failure rate > threshold (e.g. 50% in 10 seconds)
└─ └─> OPEN

OPEN (failing fast)
├─ All requests immediately return error (no downstream call)
├─ Upstream gets fast failure instead of waiting for timeout
├─ After timeout period (e.g. 30 seconds)
└─ └─> HALF-OPEN
```

```
HALF-OPEN (testing recovery)
├─ Allow small number of test requests through
├─ If they succeed: CLOSE the circuit (service recovered)
└─ If they fail:    re-OPEN the circuit (still broken)
```

Retry with Exponential Backoff and Jitter

RETRY STRATEGY

Naive retry (DO NOT DO THIS):

```
t=0s: Request → fails → retry
t=0s: Request → fails → retry    ← all 1000 clients retry at same time
t=0s: Request → fails → retry    ← thundering herd hits already-struggling
service
```

Exponential backoff:

```
Attempt 1: wait 1s
Attempt 2: wait 2s
Attempt 3: wait 4s
Attempt 4: wait 8s → give up
```

Exponential backoff + jitter (RECOMMENDED):

```
Attempt 1: wait 1s + random(0, 1s) → e.g. 1.37s
Attempt 2: wait 2s + random(0, 2s) → e.g. 3.14s
Attempt 3: wait 4s + random(0, 4s) → e.g. 5.89s
Jitter spreads retries across time – no synchronized stampede
```

PHASE 6

Reliability & Observability

6.1 SLIs, SLOs, and SLAs

Term	Definition	Example
SLI — Service Level Indicator	A measurable metric that captures one aspect of service quality	Request success rate, p99 latency, error rate
SLO — Service Level Objective	The internal target value for an SLI — your team's goal	99.9% of requests succeed; p99 latency < 200ms
SLA — Service Level Agreement	A contract with customers specifying consequences for missing SLOs	99.9% uptime or you get 10% credit on your bill
Error Budget	The allowed amount of unreliability per period (1 - SLO)	99.9% SLO = 0.1% error budget = 43.8 min/month downtime

Error Budgets in Practice

If your error budget is 43 minutes/month and you've consumed 40 minutes with 2 weeks left in the month, you freeze non-essential deployments. If you have plenty of budget remaining, you move faster. This aligns reliability work with feature work — reliability has a budget, not an infinite obligation.

6.2 The Three Pillars of Observability

Pillar	What It Tells You	Tools	Best For
Metrics	Aggregated numeric measurements over time	Prometheus, Grafana, Datadog	Dashboards, alerting, capacity planning
Logs	Discrete events with context	Elasticsearch + Logstash + Kibana (ELK)	Debugging specific errors, audit trails
Traces	The path of a request across services	Jaeger, Zipkin, AWS X-Ray	Finding where latency comes from, service dependencies

DISTRIBUTED TRACE — REQUEST FLOW

User request: GET /checkout

Trace ID: abc-123 (same ID propagated via headers across all services)

```

|— API Gateway      [0ms ————— 320ms]
  |— Auth Service   [5ms — 45ms]
  |— Cart Service    [50ms ————— 150ms]
    |— Redis Cache   [55ms — 60ms] (cache hit)
    |— Product DB    [65ms ————— 140ms] ← SLOW — investigate
  |— Payment Service [160ms ————— 310ms]
    |— Stripe API     [165ms ————— 305ms] ← external latency
  
```

Without traces: you see 320ms latency but don't know who to blame.
 With traces: you see Product DB and Stripe are the bottlenecks.

6.3 Security Fundamentals

Authentication vs Authorisation

	Authentication (AuthN)	Authorisation (AuthZ)
Question	Who are you?	What are you allowed to do?
Mechanism	Password, MFA, biometric, certificate	Roles, permissions, policies
Verified by	Identity provider (IdP), JWT validation	Authorization server, policy engine (OPA)
Happens	At login / per request	After authentication on each resource access
Example	Your username and password prove you are Alice	Alice has the 'admin' role which allows deleting users

JWT — JSON Web Tokens

JWT STRUCTURE AND VALIDATION

JWT = Header.Payload.Signature (base64url encoded, dot-separated)

```

Header: { alg: 'RS256', typ: 'JWT' }
Payload: { sub: 'user123', role: 'admin', exp: 1700000000, iat: 1699996400 }
Signature: RSA_sign(header + '.' + payload, private_key)
  
```

Validation (on every API request):

1. Decode header and payload (no secret needed — it's just base64)
2. Verify signature using PUBLIC key (server never needs private key)
3. Check exp claim — is the token expired?
4. Check iss claim — was this issued by our auth server?
5. Read sub/role claims — who is this and what can they do?

Key properties:

- Stateless: server doesn't need to query a database to validate
- Cannot be revoked before expiry (use short TTL + refresh tokens)

- Never store sensitive data in payload (it's readable by anyone)

PHASE 7

Case Studies

7.1 URL Shortener — Full Design

Requirements

Functional: Given a long URL, generate a short code (e.g. bit.ly/xK3p). Given a short code, redirect to the original URL. Optional: custom aliases, expiry, click analytics.

Non-functional: 100M URLs created/day. 10B redirects/day. Redirect latency < 10ms. High availability (99.99%). URLs never change once created.

Scale Estimation

BACK-OF-ENVELOPE CALCULATION

Write QPS: $100M / 86400 \text{ seconds} \approx 1,200 \text{ writes/sec}$

Read QPS: 10B / 86400 seconds $\approx$ 115,000 reads/sec (read:write ratio = 100:1)

Daily storage: $100\text{M} \times 307 \text{ bytes} \approx 30 \text{ GB/day}$

5-year storage: $30 \text{ GB} \times 365 \times 5 \approx 54 \text{ TB}$

Key Design Decisions

URL SHORTENER ARCHITECTURE

Short Code Generation – Option A: Counter + Base62 encoding

Problem: counter is a bottleneck and SPOF, reveals enumeration

Redirect: 301 vs 302

301 (Permanent): browser caches forever → fewer requests to our servers

but breaks click analytics (cached redirects bypass us)

Choose 302 if analytics matter, 301 for bandwidth savings

```
Redis for hot codes (in-memory cache), Cassandra for persistence
```

No joins needed, no complex queries - pure KV access pattern

```
Cache strategy: cache top 20% of URLs (those drive 80% of traffic)
Cache-aside with TTL based on last access time
```

7.2 Social Media Feed — Fanout Design

The Core Problem

When a user with 50 million followers posts a tweet, how do you make that tweet appear in all 50 million followers' feeds with low latency? This is the fanout problem.

Strategy	How It Works	Pro	Con	Best For
Fanout-on-Write (Push)	On post, pre-compute and write to all followers' feed caches	$O(1)$ read — feed is ready instantly	$O(n)$ write — writing to 50M feeds takes time	Users with few followers; read-heavy workloads
Fanout-on-Read (Pull)	On feed request, query all followed users' recent posts and merge	$O(1)$ write	$O(n)$ read — must query every followed user's posts	Users with many followers (celebrities)
Hybrid (Twitter's approach)	Push to most followers; celebrities' posts pulled separately at read	Balances both extremes	Complex to implement	Production systems with power users

HYBRID FANOUT ARCHITECTURE

```
Celebrity post (>10k followers) → Kafka → NOT pre-computed
Regular user post (<10k followers) → Kafka → Fanout workers → write to followers'
feed in Redis
```

Feed Read (for user Alice):

1. Fetch Alice's pre-computed feed from Redis (regular users' posts)
2. Fetch recent posts from all celebrities Alice follows (on-the-fly)
3. Merge and sort by time, apply ranking model
4. Return top 20 posts

Redis feed structure: Sorted Set per user

Key: feed:{user_id}

Members: post ids sorted by timestamp

Trim to last 1000 entries — users rarely scroll that far

7.3 Distributed Cache (Redis Design)

Redis Data Structures and Use Cases

Structure	Operations	Use Case
String	GET, SET, INCR, EXPIRE	Session store, counters, simple caching, rate limiting
Hash	HGET, HSET, HGETALL	User profiles, object caching (avoid serialisation overhead)
List	LPUSH, RPOP, LRange	Message queues, activity feeds, recent items
Set	SADD, SMEMBERS, SINTERSTORE	Unique visitors, tags, social graph (following/followers)
Sorted Set	ZADD, ZRange, ZRangeByScore	Leaderboards, rate limiting, priority queues, timelines
Bitmap	SETBIT, GETBIT, BITCOUNT	Daily active users (each bit = one user ID)
HyperLogLog	PFADD, PFCOUNT	Approximate unique counts (< 1% error, uses 12KB regardless of set size)

REDIS CLUSTER ARCHITECTURE

Redis Cluster: 3 primary + 3 replica shards (minimum for production)

```
[Primary 0] ↔ [Replica 0]    handles hash slots 0-5460
[Primary 1] ↔ [Replica 1]    handles hash slots 5461-10922
[Primary 2] ↔ [Replica 2]    handles hash slots 10923-16383
```

Key routing: $\text{CRC16}(\text{key}) \% 16384 \rightarrow$ determines which shard
All clients know the slot map $\rightarrow$ route directly to correct node

Failure: if Primary 0 fails, Replica 0 is promoted to primary
Clients are redirected via MOVED or ASK responses

Hash tags: {user}.profile and {user}.settings always co-located
Use {} to force keys to the same slot for multi-key operations

You have reached the end of the System Design coursework.

The only way to get good at system design is to design systems — then defend your choices.